In-Silico Target Identification in bovine mastitis causing pathogens

Major: *Staphylococcus aureus, Escherichia coli, Streptococcus uberis, Klebsiella pneumoniae.*

Data Processing Workflow

NCBI proteomes → OrthoFinder → core orthogroups → CD-HIT → bovine/human non-homology → DEG → TTD → ProtParam → annotation → Gram classification → PSORTb

Step 1: Download Proteome of bacteria

```
#  Download the protein FASTA files (.faa) from NCBI OR or NCBI Datasets in your system

https://ftp.ncbi.nlm.nih.gov/genomes/genbank/bacteria/
```

Step 1.2: Prepare Working Directories

Open MobaXterm and connect to your Linux server.

```
# 1. Create organized workshop directory tree
mkdir -p {02_Annotation,03_TargetHunt_Core,04_Druggability,results}

# examine the created folder

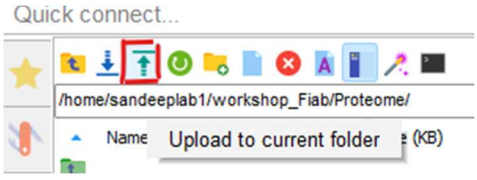
ls
```

Output:

```
02_Annotation  03_TargetHunt_Core  04_Druggability  Database  Proteome  results
```

Step 1.3: Upload proteome files on server

1. Files are downloaded on your Windows computer
2. In the left side of the MobaXterm is SFTP file browser, navigate to your Workshop_Fiab/Proteome directory.
3. Click on the highlighted arrow, it says upload to current foler, use it to upload the files in .faa in the Proteome folder
4. Wait for the upload to finish.



Step 1.4: Check uploaded FASTA files

Run the command from sandeeplab1@bioinfo:~/workshop_Fiab$

```
# Count the total .faa files

find Proteome/ -maxdepth 1 -type f -name "*.faa" | wc -l
```

Output:

```
4
```

Step 1.5: Run Orthofinder

OrthoFinder is a bioinformatics tool used to identify orthologous genes, orthogroups, and evolutionary relationships among proteins from different organisms or strains.

```
# Run orthfinder
/Bioinfo/OrthoFinder/orthofinder.py -f Proteome -t 15 -a 15
```

Run the command from the parent directory containing the Proteome folder, not from inside the Proteome folder.

Outcome: OrthoFinder folder with in the Proteome directory.

Step 2. Verification of OrthoFinder Results

Objective: To locate the OrthoFinder results directory and verify the required output files before performing pangenome analysis.

Step 2.1. Locate the OrthoFinder results directory

Identify the results folder OrthoFinder generates after the analysis.

Command:

```
# List OrthoFinder result folders
ls -d Proteome/OrthoFinder/Results_*/
```

Expected output (example):

```
Proteome/OrthoFinder/Results_Oct15/
```

Step 2.2. Check the generated output files

To confirm that OrthoFinder generated the files required for further analysis.

Commands:

```
# List files and directories in the results folder
ls Proteome/OrthoFinder/Results_*/
```

Expected output

```
(base) sandeeplab1@bioinfo:~/workshop_Fiab$ ls Proteome/OrthoFinder/Results_Oct15/
Citation.txt                    Orthogroups                          Resolved_Gene_Trees
Comparative_Genomics_Statistics Orthogroup_Sequences                 Single_Copy_Orthologue_Sequences
Gene_Duplication_Events         Orthologues                          Species_Tree
Gene_Trees                      Phylogenetically_Misplaced_Genes     WorkingDirectory
Log.txt                         Phylogenetic_Hierarchical_Orthogroups
MultipleSequenceAlignments      Putative_Xenologs
(base) sandeeplab1@bioinfo:~/workshop_Fiab$
```

Step 2.3. Verify the orthogroup table

To confirm that Orthogroups.tsv contains data and has the expected columns for the nine species.
Commands:

```
# Locate the orthogroup table

find Proteome/OrthoFinder/Results_*/ -name Orthogroups.tsv

# Display the first three rows of the orthogroup table

head -n 3 Proteome/OrthoFinder/Results_*/Orthogroups/Orthogroups.tsv

# Count the number of columns

awk -F '\t' 'NR==1 {print NF}' Proteome/OrthoFinder/Results_*/Orthogroups/Orthogroups.tsv
```

Expected result:

- The first command locate the Orthogroup table
- The second command displays orthogroup IDs and corresponding genes.
- The third command prints 5 (one orthogroup column + 4 species columns).

Step 2.4. Count the total orthogroups

To determine the number of orthogroups identified by OrthoFinder.
Command:

```
# Count orthogroups excluding the header

awk 'END {print NR-1}' Proteome/OrthoFinder/Results_*/Orthogroups/Orthogroups.tsv
```

Result: Displays the total number of orthogroups identified across the nine species.

Step 2.5. Verify orthogroup sequence files

To confirm that the orthogroup protein sequence files were generated.
Command:

```
# Check the orthogroup sequence directory

ls Proteome/OrthoFinder/Results_*/Orthogroup_Sequences/ | head
```

Expected output (example):

```
OG0000000.fa

OG0000001.fa
```

These FASTA files contain protein sequences belonging to individual orthogroups.

Step 2.6. Confirm the input file for pangenome analysis

To verify that the required orthogroup table is available before proceeding to Step 3.

Command:

```
# Verify that Orthogroups.tsv exists and is not empty

test -s Proteome/OrthoFinder/Results_*/Orthogroups/Orthogroups.tsv && echo "Orthogroups.tsv OK"
```

Expected output:

```
Orthogroups.tsv  v OK
```

Step 3. Pangenome Analysis Using Python

Objective: To identify core, accessory, and unique orthogroups from OrthoFinder results, generate a presence/absence matrix, and visualize pangenome composition.

Run the following steps sequentially from the directory containing the Proteome folder.

3.1. Install the required Python libraries

Pandas processes the orthogroup table, while matplotlib generates plots.

```
# Install libraries for data analysis and visualization

python -c "import pandas as pd; import matplotlib; print('pandas:', pd.__version__); print('matplotlib:', matplotlib.__version__)"
```

Output:

```
pandas: 1.3.5

matplotlib: 3.6.3
```

Step 3.2. Start Python and load the orthogroup table

To import the OrthoFinder orthogroup table containing gene-family information for the nine species.

Commands:

```
# Start Python

python 3
```

Output:

```
>>>
```

Step 3.3. The script helps us identify:

1. The total number of orthogroups identified by OrthoFinder.
2. The total number of species included in the analysis.
3. The names of all species represented in the orthogroup file.
4. Whether the input file can be read successfully.

Command:

```
# Import panda

import pandas as pd

# OrthoFinder output file

orthogroups_file = "./Proteome/OrthoFinder/Results_Oct27/Orthogroups/Orthogroups.tsv"

# Read file

df = pd.read_csv(orthogroups_file, sep="\t")

# Display information

print("\n===== INPUT FILE CHECK =====")

print("Number of orthogroups:", len(df))

print("Number of species:", len(df.columns) - 1)

print("\nSpecies names:")

for species in df.columns[1:]:

    print(species)
```

Input file: **Orthogroups.tsv**

Expected output:

```
Number of orthogroups: 3260
Number of species: 4
Species names:
Species_A
```

Step 3.2. To classify orthogroups into core, accessory, and unique categories based on their presence or absence across the analyzed species.

1. Identify species columns: Select the species columns from the OrthoFinder Orthogroups.tsv file, excluding the orthogroup ID column.
2. Generate a presence/absence matrix: Convert gene information into binary values:

- 1 = Orthogroup is present in a species.
- 0 = Orthogroup is absent in a species.

3. Identify core orthogroups: orthogroups are present in all analyzed species, represent conserved gene families.
4. Identify accessory orthogroups: Ppresent in at least two species but not all species.
5. Identify unique orthogroups: Present in only one species. May represent species-specific gene families.
6. Calculate species-wise unique orthogroups: Count the number of unique orthogroups belonging to each species to understand species-specific gene family distributions.
7. Save the results: Generate separate CSV files containing the presence/absence matrix and the core, accessory, and unique orthogroups for further analysis**.**

Command:

```
# Identify species columns
species_cols = df.columns[1:]
# Convert gene presence/absence into 1 and 0
binary_df = df.copy()
binary_df[species_cols] = df[species_cols].notna().astype(int)
# Save presence/absence matrix
binary_df.to_csv("Ortho_pangenome_matrix.csv", index=False)
# Count species containing each orthogroup
presence_count = binary_df[species_cols].sum(axis=1)
num_species = len(species_cols)
# Classify orthogroups
core = binary_df[presence_count == num_species]
accessory = binary_df[(presence_count >= 2) &
                      (presence_count < num_species)]
unique = binary_df[presence_count == 1]
# Count unique orthogroups for each species
unique_counts = unique[species_cols].sum()
# Save results
core.to_csv("Ortho_core_genes.csv", index=False)
accessory.to_csv("Ortho_accessory_genes.csv", index=False)
unique.to_csv("Ortho_unique_genes.csv", index=False)
# Display summary
print("\n===== PANGENOME SUMMARY =====")
print("Core orthogroups:", len(core))
print("Accessory orthogroups:", len(accessory))
print("Unique orthogroups:", len(unique))
print("\n===== UNIQUE ORTHOGROUPS PER SPECIES =====")
print(unique_counts.to_string())
# Check results
print("\n===== VALIDATION =====")
print("All orthogroups classified:",
    len(core) + len(accessory) + len(unique) == len(df))
```

Expected output:

```
===== PANGENOME SUMMARY =====
>>> print("Core orthogroups:", len(core))
Core orthogroups: 636
>>> print("Accessory orthogroups:", len(accessory))
Accessory orthogroups: 2326
>>> print("Unique orthogroups:", len(unique))
Unique orthogroups: 298
>>>
>>> print("\n===== UNIQUE ORTHOGROUPS PER SPECIES =====")

===== UNIQUE ORTHOGROUPS PER SPECIES =====
>>> print(unique_counts.to_string())
Escherichia_coli_K-12     64
Klebsiella_pneumoniae    116
Staphylococcus_aureus     94
Streptococcus_uberis      24
```

Step 3.3. To visualize the distribution of core, accessory, and unique orthogroups and compare species-specific orthogroup counts.

```
import matplotlib.pyplot as plt

# 1. Pangenome composition (core, accessory, unique)
counts = {"Core": len(core), "Accessory": len(accessory), "Unique": len(unique)}

plt.figure(figsize=(6, 5))
plt.bar(counts.keys(), counts.values(), color=["green", "blue", "red"])
plt.ylabel("Number of Orthogroups")
plt.title("Pangenome Composition")
plt.tight_layout()
plt.savefig("pangenome_composition.png", dpi=300)
plt.close()

# 2. Unique orthogroups per species
plt.figure(figsize=(8, 5))
unique_counts.plot(kind="bar", color="purple")
plt.ylabel("Number of Unique Orthogroups")
plt.title("Unique Orthogroups per Species")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.savefig("unique_orthogroups_per_species.png", dpi=300)
plt.close()

print("Both pangenome plots saved successfully!")
```

Why are these plots generated?

Plot	Purpose
Pangenome Composition	Compare core, accessory, and unique orthogroup counts
Unique Orthogroups per Species	Compare species-specific orthogroup counts

Output files

The script generates two PNG images at 300 DPI, suitable for use in reports and presentations.

1. pangenome_composition.png
2. unique_orthogroups_per_species.png

The y-axis uses Number of Orthogroups rather than Number of Genes.

Step 4. Remove redundant protein

Step 4.1: Extract core orthogroup IDs, copy their FASTA files, and merge all protein sequences, Save as 01_Extract_Core_Sequences.sh.

Command:

At the >>> prompt, type:

```
exit()
```

Press Enter.

You should return to your Linux terminal.

Command:

```
#!/usr/bin/env bash
set -euo pipefail

# Set input and output directories
ORTHO="./Proteome/OrthoFinder/Results_**/Orthogroup_Sequences"
OUT="02_Annotation"
mkdir -p "$OUT/Core_OrthoGP_Seq"

# Extract core orthogroup IDs
tail -n +2 Ortho_core_genes.csv | cut -d',' -f1 > "$OUT/Core_Ortho_Ids"

# Copy and merge core protein sequences
: > "$OUT/Core_OrthoGp_proteome.fa"

while read -r OG; do
   cp "$ORTHO/$OG.fa" "$OUT/Core_OrthoGP_Seq/"
   cat "$ORTHO/$OG.fa" >> "$OUT/Core_OrthoGp_proteome.fa"
done < "$OUT/Core_Ortho_Ids"

# Display results
echo "Core orthogroups: $(wc -l < "$OUT/Core_Ortho_Ids")"
echo "Total proteins: $(grep -c '^>' "$OUT/Core_OrthoGp_proteome.fa")"
echo "Extraction completed!"
```

Output: 04_Druggability/Core_HS_DEG_TTD.blastp

Expected output:

```
Core orthogroups: 636
Total protein sequences: 4093
Core sequence extraction completed!
```

Script 4.2: Remove Redundant Protein Sequences Using CD-HIT

Remove redundant protein sequences from the merged core orthogroup FASTA file generated by Script 4.1.

```
#!/usr/bin/env bash
set -euo pipefail

# Input and output files
INPUT="02_Annotation/Core_OrthoGp_proteome.fa"
OUTPUT="02_Annotation/Core_OrthoGp_proteome_NR1.fa"

# Remove redundant proteins at 100% identity
cd-hit -i "$INPUT" -o "$OUTPUT" -c 1.0 -n 5

# Count proteins before and after CD-HIT
echo "Proteins before CD-HIT: $(grep -c '^>' "$INPUT")"
echo "Proteins after CD-HIT: $(grep -c '^>' "$OUTPUT")"

echo "CD-HIT completed!"
```

Expected outcome:

```
Proteins before CD-HIT: 4093
Proteins after CD-HIT: 4071
CD-HIT completed!
```

Step 5: To identify core orthogroup proteins that have no detectable significant sequence similarity to bovine and human host proteins, using BLASTP.

Input: Core_OrthoGp_proteome_NR1.fa
Non-redundant core protein sequences from CD-HIT

⬇

Script 1: Bovine BLASTP
Compare core proteins against bovine proteome

⬇

Script 2: Bovine non-homolog selection
Select proteins without significant bovine BLAST hits

⬇

Script 3: Human BLASTP and filtering
Remove proteins with significant human BLAST hits

⬇

Final: Core_HS_NonHomo.fasta
Proteins without significant hits to either host

Script 5.1: BLASTP against bovine proteome

To identify core bacterial proteins showing significant sequence similarity to bovine proteins.

```
#!/usr/bin/env bash
set -euo pipefail

# Input and output
INPUT="02_Annotation/Core_OrthoGp_proteome_NR1.fa"
OUT="03_TargetHunt_Core"
mkdir -p "$OUT"

# BLASTP: identity and query coverage included
blastp -query "$INPUT" -db Database/BT \
  -out "$OUT/Core_BT.tsv" \
  -outfmt "6 qseqid pident qcovhsp evalue" \
  -evalue 1e-20 -num_threads 25

echo "Bovine BLASTP completed!"
```

Expected outcome:

```
Bovine BLASTP completed!
```

Output: 03_TargetHunt_Core/Core_BT.tsv containing significant bovine BLAST matches and their alignment statistics.,

Script 5.2: Extract bovine non-homologous proteins

Extract bovine non-homologous proteins

Purpose: Extract core bacterial proteins that have no significant BLASTP hits against the bovine proteome.

Why is this script used?

1. Identify bovine-matching proteins & excludes proteins with significant bovine BLAST hits.
2. Saves proteins without significant bovine matches into a new FASTA file.
3. Count filtered proteins: Displays the total number of retained protein sequences.

```
#!/usr/bin/env bash
set -euo pipefail

OUT="03_TargetHunt_Core"

# Identify proteins with identity >40% OR coverage >40%
awk '$2>40 || $3>40 {print $1}' "$OUT/Core_BT.tsv" |
  sort -u > "$OUT/Core_BT_Exclude.ids"

# Extract retained proteins
python3 - <<'PY'
from Bio import SeqIO

out = "03_TargetHunt_Core"
excluded = set(open(f"{out}/Core_BT_Exclude.ids").read().splitlines())

records = (r for r in SeqIO.parse(
   "02_Annotation/Core_OrthoGp_proteome_NR1.fa", "fasta"
) if r.id not in excluded)

n = SeqIO.write(records, f"{out}/Core_BT_NonHomo.fasta", "fasta")
print("Bovine-filtered proteins:", n)
PY
```

Expected output: Output: 03_TargetHunt_Core/Core_BT_NonHomo.fasta

```
Bovine non-homologous proteins: 2899
```

Script 5.3: Human Host Non-Homology Filtering

Purpose: Identify bacterial core proteins that have no significant BLASTP matches against the human proteome, after bovine non-homology filtering.

Why is this step performed?

1. To compare bovine-filtered bacterial proteins against the human proteome.
2. To identify and exclude proteins with significant human BLASTP matches.
3. To retain proteins without significant matches to either bovine or human proteins.

following criteria used:

Parameter	Cutoff
BLASTP E-value	≤ 1e-20
Sequence identity	40%
Query coverage	40%
Retain	No BLAST hit, or all reported hits have identity ≤ 40% and coverage ≤ 40%
Exclude	Any reported hit with identity > 40% or coverage > 40%

Command:

```
#!/usr/bin/env bash
set -euo pipefail

OUT="03_TargetHunt_Core"

# BLASTP against human proteome
blastp -query "$OUT/Core_BT_NonHomo.fasta" -db Database/HS \
  -out "$OUT/Core_HS.tsv" \
  -outfmt "6 qseqid pident qcovhsp evalue" \
  -evalue 1e-20 -num_threads 25

# Exclude proteins exceeding either cutoff
awk '$2>40 || $3>40 {print $1}' "$OUT/Core_HS.tsv" |
  sort -u > "$OUT/Core_HS_Exclude.ids"

# Extract final retained proteins
python3 - <<'PY'
from Bio import SeqIO

out = "03_TargetHunt_Core"
excluded = set(open(f"{out}/Core_HS_Exclude.ids").read().splitlines())

records = (r for r in SeqIO.parse(
    f"{out}/Core_BT_NonHomo.fasta", "fasta"
) if r.id not in excluded)

n = SeqIO.write(records, f"{out}/Core_HS_NonHomo.fasta", "fasta")
print("Final host-filtered proteins:", n)
PY
```

Expected outcome: 03_TargetHunt_Core/Core_HS_NonHomo.fasta

```
Final host-filtered proteins: 2859
```

Verify the final FASTA file

```
ls -lh 03_TargetHunt_Core/Core_HS_NonHomo.fasta
```

Confirm the number of proteins

```
grep -c '^>' 03_TargetHunt_Core/Core_HS_NonHomo.fasta
```

Expected Outcome:

```
2859
```

Check how many proteins were retained at each stage

Command:

```
echo "Initial proteins:"
grep -c '^>' 02_Annotation/Core_OrthoGp_proteome_NR1.fa

echo "After bovine filtering:"
grep -c '^>' 03_TargetHunt_Core/Core_BT_NonHomo.fasta

echo "After human filtering:"
grep -c '^>' 03_TargetHunt_Core/Core_HS_NonHomo.fasta
```

Expected outcome:

```
grep -c '^>' 03_TargetHunt_Core/Core_HS_NonHomo.fasta
Initial proteins:
4071
After bovine filtering:
2899
After human filtering:
2859
```

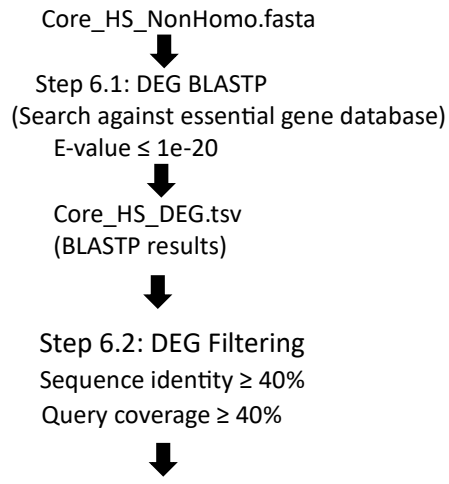
Step 6: DEG Essential Protein Screening

Purpose: Identify potentially essential bacterial proteins among the 2,859 host-filtered proteins by comparing them against the Database of Essential Genes (DEG).

Selection criteria

Parameter	Cutoff
BLASTP E-value	≤ 1e-20
Sequence identity	≥ 40%
Query coverage	≥ 40%

These criteria identify proteins homologous to DEG entries.



Core_HS_DEG.ids
(Selected unique protein IDs)



Step 6.3: FASTA Extraction
(Extract selected protein sequences)



Core_HS_DEG.fasta
(Candidate essential proteins)

Script 6.1: BLASTP against DEG

To identify host-filtered bacterial proteins with significant similarity to known essential proteins.

Command:

```
cd ~/workshop_Fiab

mkdir -p 04_Druggability

blastp -query 03_TargetHunt_Core/Core_HS_NonHomo.fasta \
 -db Database/DEGaa \
 -out 04_Druggability/Core_HS_DEG.tsv \
 -outfmt "6 qseqid pident qcovhsp evalue" \
 -evalue 1e-20 \
 -num_threads 45

echo "DEG BLASTP completed!"
```

Generated file: **04_Druggability/Core_HS_DEG.tsv**

This file contains the BLAST results with protein ID, percentage identity, query coverage, and E-value.

Expected Outcome:

```
-out 04_Druggability/Core_HS_DEG.tsv \
 -outfmt "6 qseqid pident qcovhsp evalue" \
 -evalue 1e-20 \
 -num_threads 45
```

Script 6.2: Select potential essential proteins

To retain only proteins with identity ≥ 40% and query coverage ≥ 40% against DEG entries.

Command:

```
#!/usr/bin/env bash
set -euo pipefail

OUT="04_Druggability"

# Select proteins meeting identity and coverage cutoffs
awk '$2>=40 && $3>=40 {print $1}' "$OUT/Core_HS_DEG.tsv" |
 sort -u > "$OUT/Core_HS_DEG.ids"

echo "Candidate essential proteins:"
wc -l < "$OUT/Core_HS_DEG.ids"
```

Generated file: **04_Druggability/Core_HS_DEG.ids**
This contains unique IDs of proteins passing the DEG selection criteria.
Expected Outcome:

```
Candidate essential proteins: 1621
```

Script 6.3: Extract essential protein sequences

To generate a **FASTA file** containing only the selected candidate essential proteins for subsequent screening.
Command:

```
#!/usr/bin/env bash
set -euo pipefail

# Extract candidate essential protein sequences
python3 - <<'PY'
from Bio import SeqIO

out = "04_Druggability"
ids = set(open(f"{out}/Core_HS_DEG.ids").read().splitlines())

records = (r for r in SeqIO.parse(
   "03_TargetHunt_Core/Core_HS_NonHomo.fasta", "fasta"
) if r.id in ids)

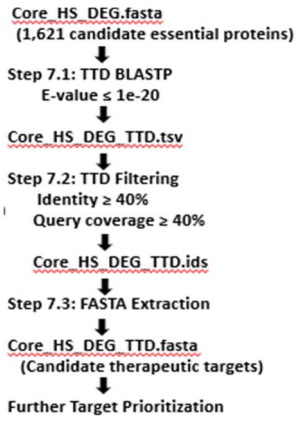
n = SeqIO.write(records, f"{out}/Core_HS_DEG.fasta", "fasta")
print("Candidate essential proteins:", n)
PY
```

Generated file: **04_Druggability/Core_HS_DEG.fasta**
This is the main FASTA file for downstream analysis.

File	Reason
Core_HS_DEG.tsv	BLAST alignment evidence and verification
Core_HS_DEG.ids	IDs of selected candidate essential proteins
Core_HS_DEG.fasta	Main input for subsequent target screening

Expected outcome: A FASTA dataset containing host-filtered bacterial proteins with significant similarity to known essential proteins in DEG, suitable for subsequent drug-target prioritization.

Step 7: Druggability Screening Against TTD



Purpose: Identify potential drug targets among your 1,621 DEG-selected candidate essential proteins by comparing them against the Therapeutic Target Database (TTD).

election criteria

Parameter	Cutoff
BLASTP E-value	≤ 1e-20
Sequence identity	≥ 40%
Query coverage	≥ 40%

Script 7.1 — BLASTP against TTD

To find candidate essential proteins with similarity to known therapeutic targets.

Command:

```
#!/usr/bin/env bash
set -euo pipefail

# Step 1: Compare DEG-selected proteins against TTD
blastp \
  -query 04_Druggability/Core_HS_DEG.fasta \
  -db Database/TTD \
  -out 04_Druggability/Core_HS_DEG_TTD.tsv \
  -outfmt "6 qseqid pident qcovhsp evalue" \
  -evalue 1e-20 \
  -num_threads 45

echo "TTD BLASTP completed!"
```

Expected Outcome: 04_Druggability/Core_HS_DEG_TTD.tsv
This contains BLAST matches with query protein ID, identity, query coverage, and E-value.

Expected Outcome:

```
TTD BLASTP completed!
```

Script 7.2 — Filter potential drug targets

To select proteins matching TTD entries at identity ≥ 40% and query coverage ≥ 40%.

Command:

```
#!/usr/bin/env bash

set -euo pipefail

# Step 2: Retain proteins passing identity and coverage cutoffs

awk '$2>=40 && $3>=40 {print $1}' \

  04_Druggability/Core_HS_DEG_TTD.tsv |

  sort -u > 04_Druggability/Core_HS_DEG_TTD.ids

# Count selected candidate drug targets

echo "Candidate drug targets:"

wc -l < 04_Druggability/Core_HS_DEG_TTD.ids
```

Output: 04_Druggability/Core_HS_DEG_TTD.ids

Contains the unique IDs of selected candidate drug targets.

Expected Outcome:

```
Candidate drug targets: 132
```

Script 7.3 — Extract drug-target protein sequences

To obtain the amino acid sequences of selected proteins for further drug-target analysis.

Command:

```
#!/usr/bin/env bash

set -euo pipefail

# Step 3: Extract selected protein sequences

python3 - <<'PY'

from Bio import SeqIO

out = "04_Druggability"

ids = set(open(f"{out}/Core_HS_DEG_TTD.ids").read().splitlines())

records = (r for r in SeqIO.parse(

    f"{out}/Core_HS_DEG.fasta", "fasta"

) if r.id in ids)

n = SeqIO.write(records, f"{out}/Core_HS_DEG_TTD.fasta", "fasta")

print("Candidate drug-target proteins:", n)

PY
```

Expected Outcome: 04_Druggability/Core_HS_DEG_TTD.fasta
This is the final TTD-screened FASTA file.
Outcome:

```
Candidate drug targets: 132
```

Files generated

File	Purpose
Core_HS_DEG_TTD.tsv	BLASTP evidence against TTD
Core_HS_DEG_TTD.ids	Selected candidate drug-target IDs
Core_HS_DEG_TTD.fasta	Selected protein sequences for downstream analysis

Step 8: ProtParam Analysis and Protein Stability Screening

Purpose: Calculate physicochemical properties of TTD-selected proteins and retain proteins predicted to be stable based on their instability index.
Selection criteria

Parameter	Criterion
Instability index	< 40
Stable proteins	Instability index < 40
Unstable proteins	Instability index ≥ 40

An instability index below 40 suggests predicted in vitro stability. It does not guarantee that the protein will be stable under all experimental conditions.

Core_HS_DEG_TTD.fasta (TTD-selected proteins)
⬇
Step 8.1: ProtParam Analysis
Calculate physicochemical protein properties
⬇
Instability Index < 40
⬇
Core_HS_DEG_TTD_ProtParam.csv (Stable protein properties)
⬇
Step 8.2: FASTA Extraction
⬇
Core_HS_DEG_TTD_Stable.fasta (Stable proteins)

Script 8.1 — Calculate ProtParam properties

To calculate physicochemical properties and identify proteins with an instability index below 40.
Start Python and load the python libraries

Command:

```
python3 - <<'PY'
from Bio import SeqIO
from Bio.SeqUtils.ProtParam import ProteinAnalysis
import pandas as pd
# Step 8.1: Calculate ProtParam properties
rows = []
for r in SeqIO.parse("04_Druggability/Core_HS_DEG_TTD.fasta", "fasta"):
    try:
        p = ProteinAnalysis(str(r.seq).replace("*", ""))
        rows.append([r.id, len(r.seq), round(p.molecular_weight(), 2),
                     round(p.isoelectric_point(), 2),
                     round(p.instability_index(), 2),
                     round(p.gravy(), 3)])
    except ValueError:
        print("Skipped:", r.id)
# Retain stable proteins (instability index < 40)
df = pd.DataFrame(rows, columns=[
    "Protein_ID", "Length", "Molecular_Weight",
    "Isoelectric_Point", "Instability_Index", "GRAVY"
])
stable = df[df["Instability_Index"] < 40]
stable.to_csv("04_Druggability/Core_HS_DEG_TTD_ProtParam.csv", index=False)
print("Total analysed:", len(df))
print("Stable proteins:", len(stable))
PY
```

Expected Outcome:

```
Total analysed: 132
Stable proteins: 88
```

Step 8.2 — Extract Stable Protein Sequences

To extract amino acid sequences of proteins with an instability index < 40, selected in Step 8.1, for further drug-target screening.

Command:

```
python3
```

Run this inside Python (>>>):

```
from Bio import SeqIO
import pandas as pd
# Step 1: Read stable protein IDs from ProtParam results
out = "04_Druggability"
ids = set(pd.read_csv(f"{out}/Core_HS_DEG_TTD_ProtParam.csv")["Protein_ID"])
# Step 2: Extract stable protein sequences
seqs = [r for r in SeqIO.parse(f"{out}/Core_HS_DEG_TTD.fasta", "fasta")
        if r.id in ids]
# Step 3: Save FASTA and display count
n = SeqIO.write(seqs, f"{out}/Core_HS_DEG_TTD_Stable.fasta", "fasta")
print("Stable proteins extracted:", n)
```

Expected Outcome:

```
Stable proteins extracted: 88
```

Generated file: 04_Druggability/**Core_HS_DEG_TTD_Stable.fasta**
Selection criterion: Instability index < 40
Outcome: A FASTA file containing 88 predicted stable protein sequences for downstream drug-target screening.

Step 9: Functional Annotation of Stable Proteins

Purpose: Recover the original protein descriptions for your 88 stable proteins from the source proteome .faa files.
We can divide your script into 2 short Python scripts. Only two final files are needed.

Script 9.1: Recover Protein Annotations

To retrieve original protein annotations for the 88 stable proteins.

Run inside Python (>>>):

Command:

```
from Bio import SeqIO
import glob
import pandas as pd
# Collect annotations from original proteomes
ann = {r.id: r.description for f in glob.glob("Proteome/*.faa")
      for r in SeqIO.parse(f, "fasta")}
# Match stable protein IDs with annotations
out = "04_Druggability"
rows = [(r.id, ann.get(r.id, "NOT_FOUND"))
       for r in SeqIO.parse(f"{out}/Core_HS_DEG_TTD_Stable.fasta", "fasta")]
# Save annotation table
pd.DataFrame(rows, columns=["Protein_ID", "Annotation"]).to_csv(
    f"{out}/Core_HS_DEG_TTD_Stable_Annotation.tsv", sep="\t", index=False)
print("Proteins:", len(rows))
print("Annotated:", sum(a != "NOT_FOUND" for _, a in rows))
```

Expected outcome: An annotation table containing 88 protein IDs and their corresponding descriptions.
Generated file: 04_Druggability/Core_HS_DEG_TTD_Stable_Annotation.tsv
Expected Outcome:

```
Total proteins processed: 88
Annotation table generated:
04_Druggability/Core_HS_DEG_TTD_Stable_Annotation.tsv
```

Check annotation count

```
print("Annotated:", sum(a != "NOT_FOUND" for _, a in rows))
```

Expected Outcome:

```
Annotated: 88
```

Check missing annotations

```
print("Not found:", sum(a == "NOT_FOUND" for _, a in rows))
```

Expected Outcome:

```
Not found: 0
```

To check whether the annotation file was generated, run:

```
import os

file = "04_Druggability/Core_HS_DEG_TTD_Stable_Annotation.tsv"

print("File exists:", os.path.isfile(file))
```

Expected Outcome:
"04_Druggability/Core_HS_DEG_TTD_Stable_Annotation.tsv"
Exit from python

```
Exit()
```

Step 10: Separate Gram-Positive and Gram-Negative Proteins

Purpose: Divide your 88 stable proteins into Gram-positive and Gram-negative groups based on their bacterial species.

Classification criteria

Gram-positive	Gram-negative
Staphylococcus aureus	Escherichia coli
Streptococcus uberis	Klebsiella pneumoniae

Script 10.1 - Extract Gram-positive proteins

To identify stable proteins belonging to Gram-positive bacteria.

Run inside Python (>>>):

```
python3 - <<'PY'
import pandas as pd
from Bio import SeqIO


out = "04_Druggability"
df = pd.read_csv(f"{out}/Core_HS_DEG_TTD_Stable_Annotation.tsv", sep="\t")
seqs = list(SeqIO.parse(f"{out}/Core_HS_DEG_TTD_Stable.fasta", "fasta"))

for name, pattern in [
   ("GramPositive", r"Staphylococcus|Streptococcus"),
   ("GramNegative", r"Escherichia|Klebsiella")
]:
   ids = set(df.loc[df["Annotation"].str.contains(pattern, case=False, na=False), "Protein_ID"])
   n = SeqIO.write([r for r in seqs if r.id in ids],
                   f"{out}/Core_HS_DEG_TTD_Stable_{name}.fasta", "fasta")
   print(name, "sequences:", n)
PY
```

Expected Outcome:

```
GramPositive sequences: 40

GramNegative sequences: 48
```

Then exit from python

```
Exit()
```

Script 10.2 — Extract Gram-negative proteins

run this command to check whether both FASTA files were generated:

Command:

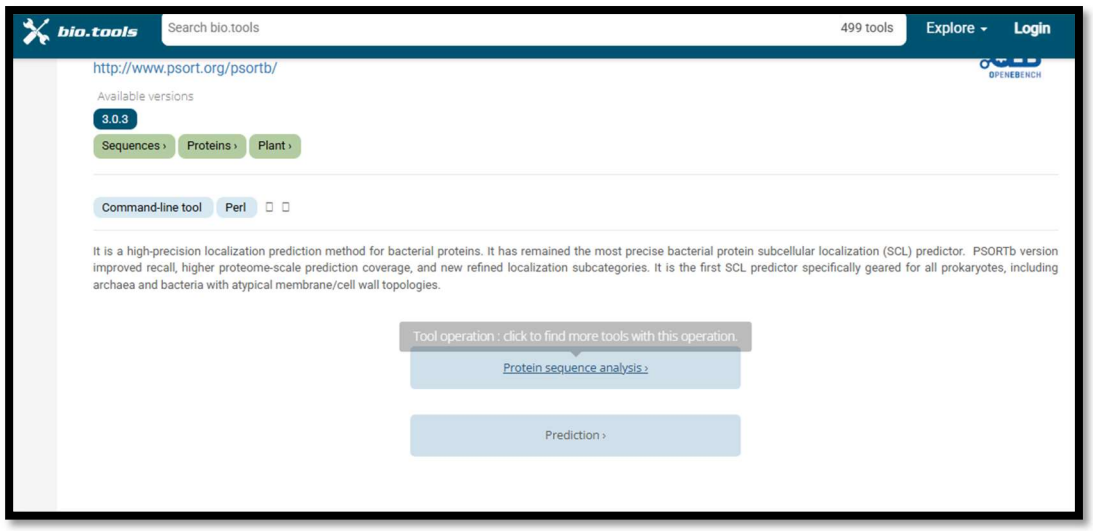
```
ls -lh 04_Druggability/Core_HS_DEG_TTD_Stable_GramPositive.fasta 04_Druggability/Core_HS_DEG_TTD_Stable_GramNegative.fasta
```

Outcome:

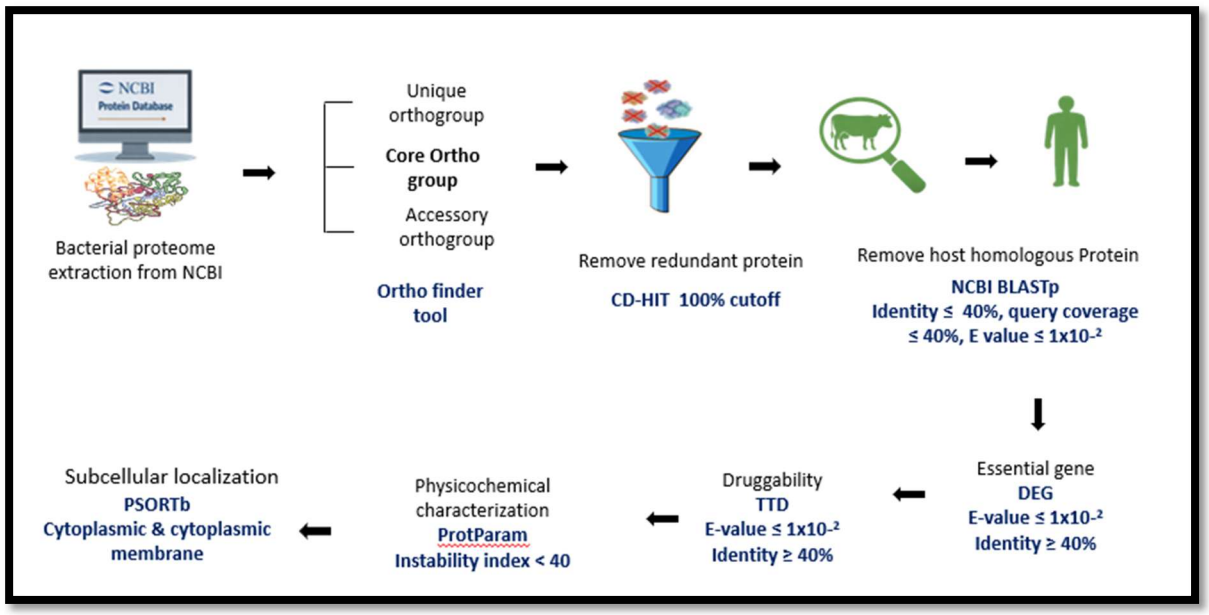
04_Druggability/Core_HS_DEG_TTD_Stable_GramPositive.fasta

Step 11: Check subcellular localization of bacterial protein

Use PSORTb web server



Work flow:


NCBI
Protein Database
Bacterial proteome extraction from NCBI
Unique orthogroup
Core Ortho group
Accessory orthogroup
Ortho finder tool
Remove redundant protein
CD-HIT 100% cutoff
Remove host homologous Protein
NCBI BLASTp
Identity ≤ 40%, query coverage ≤ 40%, E value ≤ 1x10-2
Essential gene
DEG
E-value ≤ 1x10-2
Identity ≥ 40%
Druggability
TTD
E-value ≤ 1x10-2
Identity ≥ 40%
Physicochemical characterization
ProtParam
Instability index < 40
Subcellular localization
PSORTb
Cytoplasmic & cytoplasmic membrane